

```
import torch
print(torch.__version__)
```

```
2.11.0+cu128
```

```
if torch.cuda.is_available():
    print('GPU is available')
    print(f"Using GPU: {torch.cuda.get_device_name(0)}")
else:
    print("No GPU")
```

```
GPU is available
Using GPU: Tesla T4
```

Start coding or [generate](#) with AI.

Creating Tensors

```
# Using Empty
torch.empty(2,3)
```

```
tensor([[9.5837e+24, 4.4357e-41, 9.5837e+24],
        [4.4357e-41, 1.4013e-45, 0.0000e+00]])
```

```
# Zeros and ones
```

```
a = torch.zeros(2,3)
b = torch.ones(3, 3)
```

```
print(a)
print(b)
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
```

```
# Random
torch.rand(2, 3)
```

```
tensor([[0.6593, 0.4002, 0.2211],
        [0.6330, 0.4095, 0.9537]])
```

```
# for any experimentation if you want to keep random values same everywhere then you can use
# Seed or manual_seed()
torch.manual_seed(42)
torch.rand(2,3)
```

```
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

```
# tensor: to create custom tensors
torch.tensor([[1,2,3], [4,5,6]])
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

```
torch.tensor([1, 2,3, 4, 5, 6])
```

```
tensor([1, 2, 3, 4, 5, 6])
```

```
# arange
print("Using arange: ", torch.arange(0, 10, 2))
```

```
# Linspace
print("Linspace: ", torch.linspace(0, 10, 10))
```

```
# eye: identity matrix
print('using eye: \n', torch.eye(5))
```

```
# full: shape and item value
print('using full: \n', torch.full((3, 3), 5))
```

```

Using arange: tensor([0, 2, 4, 6, 8])
Linspace: tensor([ 0.0000,  1.1111,  2.2222,  3.3333,  4.4444,  5.5556,  6.6667,  7.7778,
                  8.8889, 10.0000])
using eye:
  tensor([[1., 0., 0., 0., 0.],
          [0., 1., 0., 0., 0.],
          [0., 0., 1., 0., 0.],
          [0., 0., 0., 1., 0.],
          [0., 0., 0., 0., 1.]])
using full:
  tensor([[5, 5, 5],
          [5, 5, 5],
          [5, 5, 5]])

```

Tensor shapes

```

x = torch.tensor([[[1, 2, 3], [4, 5, 6]]])
x

```

```

tensor([[1, 2, 3],
        [4, 5, 6]])

```

```
x.shape
```

```
torch.Size([2, 3])
```

```

# to create tensor of same size as x with random values
torch.empty_like(x)

```

```

tensor([[          634001860, 4607182418800017408, 4607182418800017408],
        [4607182418800017408,             80,          96]])

```

```
torch.zeros_like(x)
```

```

tensor([[0, 0, 0],
        [0, 0, 0]])

```

```
torch.ones_like(x)
```

```

tensor([[1, 1, 1],
        [1, 1, 1]])

```

Data Type

```
x.dtype
```

```
torch.int64
```

```

# assign data type
torch.tensor([1.0, 2.0, 3.0], dtype=torch.int32)

```

```
tensor([1, 2, 3], dtype=torch.int32)
```

```

# using to() for same
x.to(torch.float32)

```

```

tensor([[1., 2., 3.],
        [4., 5., 6.]])

```

Arithmetic Operations

```

# addition
print(x)
x + 2

```

```

tensor([[1, 2, 3],
        [4, 5, 6]])
tensor([[3, 4, 5],
        [6, 7, 8]])

```

```

# subtraction
x-2

```

```

tensor([[ -1,  0,  1],
        [ 2,  3,  4]])

```

```
# multiplication
x * 3
```

```
tensor([[ 3,  6,  9],
        [12, 15, 18]])
```

```
# division
(x*3)/3
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```
# int division
(x*3)//3
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

```
# mod
((x*100)//3)%2
```

```
tensor([[1, 0, 0],
        [1, 0, 0]])
```

```
# power
x **2
```

```
tensor([[ 1,  4,  9],
        [16, 25, 36]])
```

▼ Element wise operations on 2 tensors

```
a = torch.rand(2, 3)
b = torch.rand(2, 3)
```

```
print(a)
print(b)
```

```
tensor([[0.2566, 0.7936, 0.9408],
        [0.1332, 0.9346, 0.5936]])
tensor([[0.8694, 0.5677, 0.7411],
        [0.4294, 0.8854, 0.5739]])
```

```
# add
a + b
```

```
tensor([[1.1260, 1.3614, 1.6819],
        [0.5626, 1.8200, 1.1675]])
```

```
# sub
a-b
```

```
tensor([[ -0.6128,  0.2259,  0.1997],
        [-0.2962,  0.0492,  0.0197]])
```

```
a*b
```

```
tensor([[0.2231, 0.4506, 0.6972],
        [0.0572, 0.8275, 0.3407]])
```

```
a/b
```

```
tensor([[0.2951, 1.3980, 1.2694],
        [0.3102, 1.0555, 1.0343]])
```

```
a**b
```

```
tensor([[0.3065, 0.8770, 0.9558],
        [0.4208, 0.9419, 0.7413]])
```

```
c = torch.tensor([1, -2, -3, 4])
```

```
torch.abs(c)
```

```
tensor([1, 2, 3, 4])
```

```
# negative
torch.neg(c)
```

```
tensor([-1,  2,  3, -4])
```

```
d = torch.tensor([1.3, 3.4, 5.2, 9.19])
d
```

```
tensor([1.3000, 3.4000, 5.2000, 9.1900])
```

```
# round
torch.round(d)
```

```
tensor([1., 3., 5., 9.])
```

```
torch.ceil(d)
```

```
tensor([ 2.,  4.,  6., 10.])
```

```
torch.floor(d)
```

```
tensor([1., 3., 5., 9.])
```

```
torch.clamp(d, min= 2, max = 6)
```

```
tensor([2.0000, 3.4000, 5.2000, 6.0000])
```

▼ Reduction function

```
e = torch.randint(size = (2, 3), low = 0, high = 10)
e
```

```
tensor([[7, 6, 9],
        [6, 3, 1]])
```

```
torch.sum(e)
```

```
tensor(32)
```

```
torch.sum(e, dim = 0)
```

```
tensor([13,  9, 10])
```

```
torch.sum(e, dim = 1)
```

```
tensor([22, 10])
```

```
e = e.to(torch.float32)
```

```
torch.mean(e)
```

```
tensor(5.3333)
```

```
torch.median(e)
```

```
tensor(6.)
```

```
torch.max(e)
```

```
torch.min(e)
```

```
tensor(1.)
```

```
print(torch.prod(e))
print(torch.std(e))
print(torch.var(e))
print(torch.argmax(e)) # ----> position of largest element
print(torch.argmin(e)) # -----> smallest element
```

```
tensor(6804.)
tensor(2.8752)
tensor(8.2667)
tensor(2)
tensor(5)
```

matrix operations

```
f = torch.randint(size=(2, 3), low = 0, high = 10)
g = torch.randint(size=(3, 2), low = 0, high = 10)
print(f)
print(g)
```

```
tensor([[9, 6, 2],
        [0, 6, 2]])
tensor([[7, 9],
        [7, 3],
        [3, 4]])
```

```
# matrix multiplication
torch.matmul(f, g)
```

```
tensor([[111, 107],
        [ 48,  26]])
```

```
vector1 = torch.tensor([1, 2])
vector2 = torch.tensor([3, 4])
```

```
# dot product of vectors
torch.dot(vector1, vector2)
```

```
tensor(11)
```

```
# transpose
torch.transpose(f, 0, 1)
```

```
tensor([[9, 0],
        [6, 6],
        [2, 2]])
```

```
h = torch.randint(size = (3, 3), low = 0, high = 10, dtype= torch.float32)
h
```

```
tensor([[4., 8., 8.],
        [6., 0., 0.],
        [0., 0., 1.]])
```

```
# determinant
torch.det(h)
```

```
tensor(-48.)
```

```
# inverse
torch.inverse(h)
```

```
tensor([[ 0.0000,  0.1667,  0.0000],
        [ 0.1250, -0.0833, -1.0000],
        [ 0.0000,  0.0000,  1.0000]])
```

Comparison operations

```
i = torch.randint(size = (2, 3), low = 0, high = 10)
j = torch.randint(size = (2, 3), low = 0, high = 10)

print(i)
print(j)
```

```
tensor([[3, 0, 1],
        [1, 7, 9]])
tensor([[4, 3, 8],
        [9, 3, 7]])
```

```
#greater than
print(i > j)
```

```
# less than
print(i < j)
```

```
# equal
print(i == j)
```

```
# greater than ewual to and less than equal to
```

```
print(i >= j)
print(i <= j)
```

```
tensor([[False, False, False],
        [False, True, True]])
tensor([[ True, True, True],
        [ True, False, False]])
tensor([[False, False, False],
        [False, False, False]])
tensor([[False, False, False],
        [False, True, True]])
tensor([[ True, True, True],
        [ True, False, False]])
```

▼ Special functions

```
k = torch.randint(size = (2, 3), low = 0, high = 10)
k
```

```
tensor([[8, 1, 4],
        [1, 6, 3]])
```

```
# log
print(torch.log(k))
```

```
# Exp
print(torch.exp(k))
```

```
# sqrt
print(torch.sqrt(k))
```

```
# sigmoid
print(torch.sigmoid(k))
#softmax and relu need float values
```

```
tensor([[2.0794, 0.0000, 1.3863],
        [0.0000, 1.7918, 1.0986]])
tensor([[2.9810e+03, 2.7183e+00, 5.4598e+01],
        [2.7183e+00, 4.0343e+02, 2.0086e+01]])
tensor([[2.8284, 1.0000, 2.0000],
        [1.0000, 2.4495, 1.7321]])
tensor([[0.9997, 0.7311, 0.9820],
        [0.7311, 0.9975, 0.9526]])
```

▼ Inplace operations

```
m = torch.rand(2, 3)
n = torch.rand(2, 3)
# so if we try to add these 2 new memory space will get created for new output tensor
# to reduce this extra space we use inplace operations
```

```
m.add_(n)
```

```
# now value is stores in m itself so no extra space is occupies
```

```
tensor([[1.3918, 0.4683, 0.8345],
        [0.1747, 0.2895, 0.8496]])
```

▼ Copying a tensor

```
a = torch.rand(2, 3)
a
```

```
tensor([[0.0624, 0.1816, 0.9998],
        [0.5944, 0.6541, 0.0337]])
```

```
b = a
b
```

```
# but this assignment operator is not always efficient
# as further chnage in a will also going to apply to b
# if you check id(a) and id(b) both will give same address
```

```
tensor([[0.0624, 0.1816, 0.9998],
        [0.5944, 0.6541, 0.0337]])
```

```
b = a.clone()
```

```
a[0][1] = 100
```

a

```
tensor([[6.2394e-02, 1.0000e+02, 9.9980e-01],  
        [5.9444e-01, 6.5408e-01, 3.3658e-02]])
```

b

```
tensor([[0.0624, 0.1816, 0.9998],  
        [0.5944, 0.6541, 0.0337]])
```

Tensor operations on GPU

```
torch.cuda.is_available()
```

```
True
```

```
device = torch.device('cuda')
```

```
# creating new tensor on GPU  
torch.rand((2, 3), device = device)
```

```
tensor([[0.6130, 0.0101, 0.3984],  
        [0.0403, 0.1563, 0.4825]], device='cuda:0')
```

```
# moving an existing tensor on GPU  
a = torch.rand(2, 3)  
a  
# '''Right now it is on cpu so lets move it to GPU'''
```

```
tensor([[0.0766, 0.8460, 0.3624],  
        [0.3083, 0.0850, 0.0029]])
```

```
b = a.to(device = device)
```

b + 5

```
tensor([[5.0766, 5.8460, 5.3624],  
        [5.3083, 5.0850, 5.0029]], device='cuda:0')
```

Let's test why GPU and not CPU

```
import time  
  
size = 10000  
  
# time on CPU  
m1_cpu = torch.rand(size, size)  
m2_cpu = torch.rand(size, size)  
  
start_time = time.time()  
res = torch.matmul(m1_cpu, m2_cpu)  
cpu_time = time.time() - start_time  
  
print(f"CPU time: {cpu_time:.4f} seconds")  
  
# time on GPU  
  
m1_gpu = torch.rand(size, size, device='cuda')  
m2_gpu = torch.rand(size, size, device='cuda')  
  
start_time = time.time()  
res = torch.matmul(m1_gpu, m2_gpu)  
gpu_time = time.time() - start_time  
  
print(f'GPU time: {gpu_time:.4f} seconds')  
  
print(f'GPU is {cpu_time/gpu_time:.4f} times faster than CPU')
```

```
CPU time: 14.9567 seconds  
GPU time: 0.0215 seconds  
GPU is 696.5996 times faster than CPU
```



Reshaping tensors

```
a = torch.ones(4, 4)
a
```

```
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
```

```
# reshape
a.reshape(2, 2, 2, 2)
```

```
tensor([[[[1., 1.],
          [1., 1.]],

        [[1., 1.],
          [1., 1.]]],

        [[[1., 1.],
          [1., 1.]],

        [[1., 1.],
          [1., 1.]]]])
```

```
# flatten
a.flatten()
```

```
tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

```
# permute
b = torch.rand(2, 3, 4)
b
```

```
tensor([[[0.4122, 0.4250, 0.8759, 0.4439],
         [0.0806, 0.4436, 0.6121, 0.0437],
         [0.0741, 0.4344, 0.2372, 0.7240]],

        [[0.5914, 0.8940, 0.0164, 0.9625],
         [0.9730, 0.8807, 0.5396, 0.9809],
         [0.4699, 0.8438, 0.6993, 0.2216]]])
```

```
b.permute(2, 0, 1)
```

```
tensor([[[0.4122, 0.0806, 0.0741],
         [0.5914, 0.9730, 0.4699]],

        [[0.4250, 0.4436, 0.4344],
         [0.8940, 0.8807, 0.8438]],

        [[0.8759, 0.6121, 0.2372],
         [0.0164, 0.5396, 0.6993]],

        [[0.4439, 0.0437, 0.7240],
         [0.9625, 0.9809, 0.2216]]]])
```

```
# unsqueeze
c = torch.rand(226, 226, 3)
c
```

```
tensor([[[0.7380, 0.2220, 0.0911],
         [0.4924, 0.8053, 0.2761],
         [0.2832, 0.4178, 0.6467],
         ...,
         [0.3251, 0.9030, 0.4915],
         [0.1714, 0.5409, 0.6963],
         [0.5336, 0.2782, 0.0058]],

        [[0.4853, 0.3868, 0.7652],
         [0.1476, 0.2767, 0.0303],
         [0.7732, 0.6391, 0.7361],
         ...,
         [0.7439, 0.4041, 0.5190],
         [0.7462, 0.3908, 0.6287],
         [0.7673, 0.2369, 0.3067]],

        [[0.1608, 0.5859, 0.5309],
         [0.1025, 0.6762, 0.5447],
         [0.7072, 0.4749, 0.2285],
         ...,
         [0.6056, 0.6704, 0.6308],
         [0.8290, 0.3285, 0.8012],
```



```
[0.5671, 0.1456, 0.2532]],
...,
[[0.8085, 0.8486, 0.6271],
 [0.5844, 0.0554, 0.1524],
 [0.8323, 0.7789, 0.6022],
 ...,
 [0.1503, 0.6582, 0.3925],
 [0.3415, 0.3154, 0.1270],
 [0.4482, 0.5420, 0.1042]],
[[0.2911, 0.6025, 0.8988],
 [0.1988, 0.0087, 0.0996],
 [0.2957, 0.7868, 0.8480],
 ...,
 [0.5833, 0.4855, 0.8119],
 [0.5764, 0.2581, 0.0266],
 [0.8158, 0.0066, 0.8933]],
[[0.1688, 0.5834, 0.2354],
 [0.6854, 0.1688, 0.3886],
 [0.9992, 0.4439, 0.4151],
 ...,
 [0.1846, 0.6561, 0.8667],
 [0.0105, 0.1615, 0.3272],
 [0.7911, 0.5433, 0.4874]]])
```

```
c.unsqueeze(dim = 0).shape
```

```
torch.Size([1, 226, 226, 3])
```

```
# squeeze
d = torch.rand(1, 20)
d.shape
```

```
torch.Size([1, 20])
```

```
d.squeeze(0).shape
```

```
torch.Size([20])
```

▼ numpy and pytorch tensor conversions

```
import numpy as np

a = torch.tensor([1, 2, 3, 4])
type(a)
```

```
torch.Tensor
```

```
b = a.numpy()
print(b)
print(type(b))
```

```
[1 2 3 4]
<class 'numpy.ndarray'>
```

```
c = np.array([1, 2, 3, 4])
type(c)
```

```
numpy.ndarray
```

```
torch.from_numpy(c)
```

```
tensor([1, 2, 3, 4])
```

Start coding or [generate](#) with AI.

